Date. / /

⇒ Longest Increasing Subsequence (LIS)

Given: Array of integers
Goal: Length of longest increasing subsequence

O(n^2) DP:

```
for (int i=1; i<n; i++)
  for (int j=0; j<i; j++)
    if (arr[i] > arr[j])
      dp[i] = max(dp[i], dp[j]+1);
```

Optimized O(n log n) using binary search

⇒ Matrix Chain Multiplication (MCM)

Given: Matrix dimensions [40, 20, 30, 10, 30]
Goal: Minimum cost to multiply mem

Recursion with DP:

```
for 'k = i to j-1;
 cost = dp[i][k] + dp[k+1][j] + arr[i-1] * arr[k] *
                                        arr[j];
```

⇒ DP on Grids

Goal: Find paths, min/max sum from (0,0) to (n-1, m-1)

Formula:

```
dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1]
```

Date. / /

=> DP on Trees

Used In: Tree diameter, node values, etc.

Recursive DP with postorder traversal

=> Digit DP (Advanced)

Used For: Count numbers with certain properties within a range

Parameters:

- pos: index in digit
- tight: is prefix same as original?
- sum: current sum or count

Summary Table

Problem	Time Complexity	DP Type
0/1 Knapsack	$O(n * W)$	2D Table
Subset Sum	$O(n * Sum)$	Boolean DP
LCS	$O(n * m)$	2D Table
LIS	$O(n \log n)$	Optimized
MCM	$O(n^3)$	DP Partition
DP on Grid	$O(n * m)$	2D Grid

Page No

Date. / /

Graphs

A graph is a non-linear data structure consisting of
- Vertices (nodes)
- Edges (connections)

Types:

- Directed or Undirected
- Weighted or Unweighted
- Cyclic or Acyclic
- Connected or Disconnected

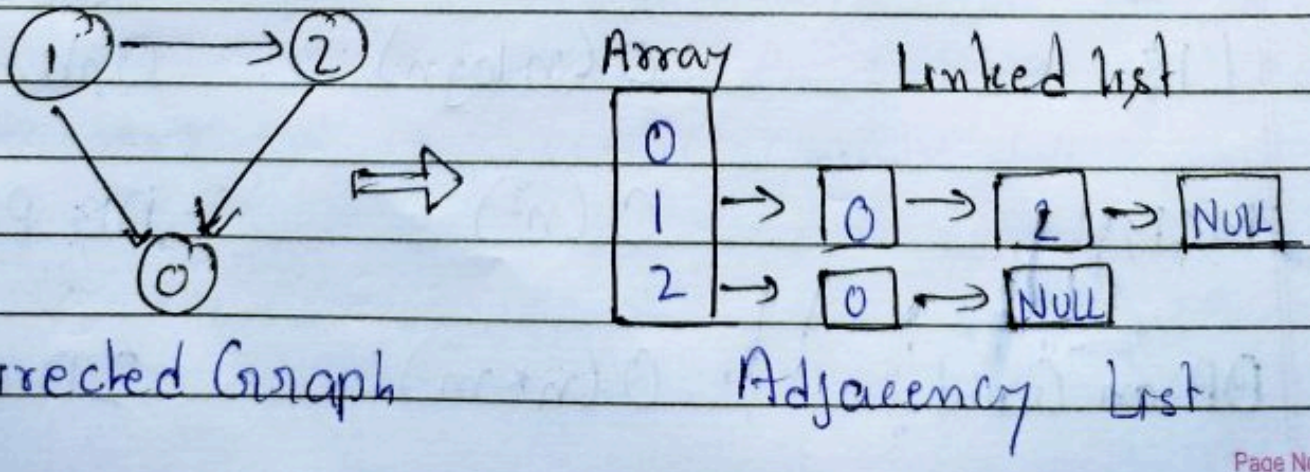
Graph Representation

1. Adjacency List

- An array / List where each index holds a list of neighbours
- Space Efficient: O(V+E)

Example

```
List<List<Integer>> adj = new ArrayList<>();
for (int i=0 ; i<V ; i++) adj.add(new ArrayList<>());
adj.get(0).add(1) ; // edge 0 → 1
```



Directed Graph Adjacency List

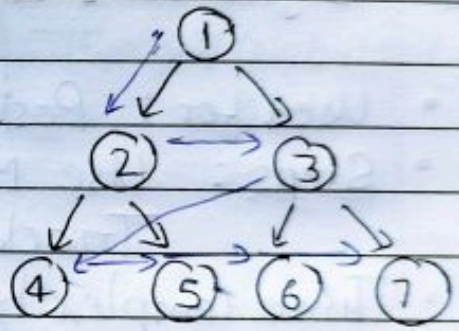
Date. __/__/____

2. Adjacency Matrix :-

- A 2D array graph[V][V]
- graph[i][j] = 1 if edge from i to j exists
- Space : $O(V^2)$

Breadth - First Search (BFS)

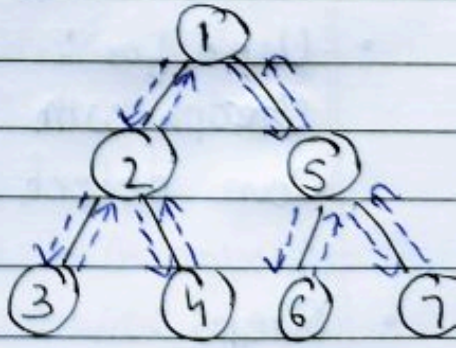
- Level by - level traversal
- Uses Queue
- Good for finding shortest path in unweighted graphs
- Time Complexity : $O(V+E)$



- Traverse through one level of children nodes, then traverse through the level of grand children nodes (and so on...)

Output : 1, 2, 3, 4, 5, 6, 7

Depth - First Search (DFS)



- Goes deep before backtracking
- Uses Stack or Recursion
- Useful for components, cycles, etc

- Traverse through left subtree(s) first, then traverse through the right subtree (s)

Output : 1, 2, 3, 4, 5,

- Time Complexity : $O(V+E)$

Date. __/__/____

Cycle Detection

1. In Undirected Graph:
   - Use DFS + parent tracking
   - If a visited node is reached and not the parent → cycle

2. In Directed Graph:
   - DFS + recursion stack
   - If a node is revisited in recursion → cycle

Dijkstra's Algorithm (Shortest Path)

- Used for: Positive weight edges only
- Steps: Use Min-Heap (Priority Queue)
  Track shortest distance from source
- Time Complexity: $O((V+E)\log V)$

Bellman-Ford Algorithm

- Used for:
  Graphs with negative weights
  Can detect negative cycles

- Steps:
  Relax all edges V-1 times
  One more pass to check for negative cycles

- Time: $O(V \times E)$

Date. __/__/____

Topological Sort

Used for Directed Acyclic Graphs (DAG)

Kahn's Algorithm (BFS-based)

- Count indegrees
- Use Queue for nodes with 0 indegree
- Process & reduce Indegree of neighbours
- Time :- $O(V+E)$

Disjoint Set (Union-Find)

- Used in:
  - Cycle detection
  - Kruskal's MST
- Operations:

  Find (x) : find parent

  Union (x, y) : combine sets

Time : $O(\alpha(N)) \approx O(1)$

- Optimization:
- Path compression
- Union by Rank

Minimum Spanning Tree (MST)

Spanning Tree ⇒ covers all nodes with minimum total edge weight

Kruskal's Algorithm:

- Greedy + Union Find
- Sort edges by weight
- Add edge if it doesn't create a cycle

Date. / /

Time : $O(E \log E)$

Prim's Algorithm:

- Use Min-Heap (Priority Queue)
- Start from any node, add shortest edge to MST

Time : $O((V+E) \log V)$

Summary Table:-

Topic	Time Complexity	Notes
BFS / DFS	$O(V+E)$	BFS uses queue, DFS uses Stack
Dijkstra	$O((V+E) \log V)$	No negative weights
Bellman-Ford	$O(V \times E)$	Can handle negative weights
Kahn's Topo Sort	$O(V+E)$	BFS-based, for DAGs
Union Find (Disjoint set)	$O(\alpha(N))$	For Kruskal, Cycle detection
Kruskal's MST	$O(E \log E)$	Greedy using Sorted edges
Prim's MST	$O((V+E) \log V)$	Greedy using Priority Queue

Sliding Window & Two Pointers

What is Sliding Window?

Sliding Window is a technique for optimizing problems involving arrays or strings by using a moving window (subarrays / substring) of elements

Types of Windows

1. Fixed-size Window:- Window of fixed size k slides across
2. Variable-size Window:- Size of window changes based on conditions (like distinct chars, sum

What is Two Pointers?

Two Pointer is a technique where you use two indices (pointers) to traverse a squence to find a condition or pattern (e.g., sorted array, substring

1. Fixed-size Window - Maximum Sum of Subarray of size k

=> Problem: Find the maximum sum of subarray of size k.

=> Java Code:

```
public static int maxSumSubarray(int[] arr, int k)
    int maxSum = 0, windowSum = 0;
```

Date. —/—/—

```
    for (int i=0 ; i<k ; i++)
        windowSum += arr[i];
    maxSum = windowSum;
    for(int i = k ; i < arr.length ; i++) {
        windowSum += arr[i] - arr[i-k];
        maxSum = Math.max(maxSum, windowSum);
    }
    return maxSum;
}
```

2. Variable-size Window – Longest Substring without Repeating characters

Problem: Given a string, find the length of the longest substring without repeating characters

Java Code:

```
public static int lengthOfLongestSubstring(String s) {
    Set<Character> set = new HashSet<>();
    int left = 0, maxLen = 0;
    for (int right = 0; right < s.length(); right++) {
        while (set.contains(s.charAt(right))) {
            set.remove(s.charAt(left++));
        }
        set.add(s.charAt(right));
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

Date. __/__/____

3. Maximum in Sliding Window (Deque Approach)

Problem: Find maximum of each subarray of size k.

Java Code:

```
public static int[] maxSlidingWindow(int[] nums, int k){
    Deque<Integer> dq = new LinkedList<>();
    List<Integer> res = new ArrayList<>();

    for(int i=0; i<nums.length; i++) {
      // Remove out of window
    if(!dq.isEmpty() && dq.peek() <= i-k) dq.poll();
      // Remove smaller values
    while(!dq.isEmpty() && nums[dq.peekLast()] <
     num[i]) dq.pollLast();
    dq.offer(i);
    if(i>=k-1) res.add(nums[dq.peek()]);
    }
    return res.stream().mapToInt(i -> i).toArray();
}
```

Time: O(n) Space: O(k)

4. Trapping Rainwater Problem

Problem: Given height of bars, calculate how much water it can trap after raining

Java Code:

```
public static int trap(int[] height) {
    int left=0, right = height.length-1;
    int leftMax=0, rightMax = 0, water=0;
```

Date. / /

```
while (left < right) {
  if (height[left] < height[right]) {
    if (height[left] >= leftMax) leftMax = height[left];
    else water += leftMax - height[left];
    left++;
  } else {
    if (height[right] >= rightMax) rightMax = height[right];
    else water += rightMax - height[right];
    right--;
  }
}
return water;
}
```

⇒ Time : O(n) Space : O(1)

⇒ Example For Trapping Rain Water:
Input : height = [0,1,0,2,1,0,1,3,2,1,2,1]
Output : 6

Summary Table

	Problem	Approach	Time	Space
1.	Max Sum Subarray (size k)	Fixed Window	O(n)	O(1)
2	Longest Substring without Repeat	Two Pointers/set	O(n)	O(n)
3.	Max in sliding Window	Deque	O(n)	O(k)
4.	Trapping Rainwater	Two Pointers	O(n)	O(1)

@Syntax Error
— Abhishek Rathor
Date. / /

Bit Manipulation

Bit Manipulation is a technique of performing operations on data at the bit level. It is highly efficient and often used for optimization in problems involving numbers, sets, toggles etc.

Common Bitwise Operators

Operator	Symbol	Example (a=5, b=3)	Result (in bits)
AND	&	a & b = 1	0101 & 0011 = 0001
OR	'	'	'a
XOR	^	a ^ b = 6	0101 ^ 0011 = 0110
NOT	~	~a = -6	~0101 = 1010 (In 2's comp.)
Left Shift	<<	a << 1 = 10	0101 << 1 = 1010
Right Shift	>>	a >> 1 = 2	0101 >> 1 = 0010

Note: Bit indexing starts from right to left, starting at 0.

1. Check if a Number is a Power of Two

A number is a power of 2 if it has only one set bit (e.g., 2 = 10, 4 = 100).

Date. / /

Java Code:

```
public static boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n-1)) == 0;
```

Explanation:

- For power of two: only one bit is 1.
- n & (n-1) removes the lowest set bit.
- So result should be 0 for power of 2

2. Count Set Bits (No. of 1s in Binary)

Java Code:

```
public static int countBits(int n) {
    int count = 0;
    while (n > 0) {
        count += n & 1;    // check last bit
        n >>= 1            // shift right
    }
    return count;
}
```

Time: O(log n)

3. Find the Single Number (All others appear twice)

Given nums where every element appears twice except one, find that one.

Java Code:

```
public static int singleNumber(int[] nums) {
    for result = 0;
```

Date. __/__/____

```
    for (int num : nums) {
        result ^= num; // XOR cancels out duplicates
    }
    return result;
}
```

Why it works?

- $a \wedge a = 0$
- $a \wedge b = b$

4. XOR of All Numbers from 1 to n

There is a pattern:

n % 4 == 0 → n

n % 4 == 1 → 1

n % 4 == 2 → n+1

n % 4 == 3 → 0

Java code:

```
public static int xorUpToN(int n) {
    if (n % 4 == 0) return n;
    if (n % 4 == 1) return 1;
    if (n % 4 == 2) return n+1;
    return 0;
}
```

Date. —/—/—

5. Find Missing Number in 0 to n

Given array of size n with elements from 0 to n. Find the missing number.

Java Code:

```
public static int missingNumber(int[] nums){
    int xor = 0;
    for(int i=0 ; i<nums.length ; i++){
        xor ^= i ^ nums[i];
    }
    return xor ^ nums.length;
}
```

6. Get ith Bit of a Number

Java Code:

```
public static int getIthBit(int n, int i){
    return (n>>i) & 1;
}
```

7. Set ith Bit

Java Code:

```
public static int setIthBit(int n, int i){
    return n | (1<<i);
}
```

8. Clear ith Bit

Java Code:

```
public static int clearIthBit(int n, int i){
    return n & ~(1<<i);
}
```

Date. / /

Examples

Example 1 : is Power Of Two (8)
Binary : 1000
8 & 7 = 0 → True

Example 2 : countBits (13)
Binary : 1101 → 3 ones

Example 3 : singleNumber ([2,2,1])
2 ^ 2 ^ 1 = 0 ^ 1 = 1

Summary Table

Problem Type	Time	Space
Power of Two	O(1)	O(1)
Count Bits	O(log n)	O(1)
Single Number (XOR)	O(n)	O(1)
XOR from 1 to n	O(1)	O(1)
Missing Number (XOR Trick)	O(n)	O(1)

Date. / /

Segment Tree & Fenwick Tree

What is Segment Tree?

A segment Tree is a binary tree structure used to perform efficient range queries and updates (e.g., sum, min, max) in logarithmic time.

Useful for :-

- Range sum queries
- Range min/max queries
- Updating values efficiently.

Basic Concept:

- Each node in the segment Tree represents a segment (range) of the input array.
- Leaf nodes store values of the array
- Internal nodes store results of queries (sum/min/max) on child segments.

Time & Space Complexities

Operation	Time
Build	$O(n)$
Query (Range)	$O(\log n)$
Update (Point)	$O(\log n)$
Space	$O(4n)$

Date. __/__/__

Example Problem: Range Sum Query

Given arr = [1,3,5,7,9,11], builds a segment tree to find sum in any range

Java Code:

```
class SegmentTree {
    int[] tree; int n

    SegmentTree(int [] arr) {
        n = arr.length;
        tree = new int [4*n]
        build (arr, 0, n-1, 0);
    }
  void build (int[] arr, int l, int r, int i) {
    if (l == r) {
        tree[i] = arr [l];
        return;
      }

    int mid = (l+r)/2;
    build (arr, l, mid, 2*i+1);
    build (arr, mid+1, r, 2*i+2);
    tree[i] = tree[2*i+1] + tree[2*i+2];
}

int query (int ql, int qr) {
    return queryUtil (0, n-1, ql, qr, 0);
}
```

Date. —/—/—

```
int queryUtil(int l, int r, int ql, int qr, int i){
  if (qr < l || ql > r) return 0;
  if (ql <= l && r <= qr) return tree[i];
  int mid = (l+r)/2;
  return queryUtil(l, mid, ql, qr, 2*i+1) +
    queryutil(mid+1, r, ql, qr, 2*i+2);
}

void update(int idx, int val){
  updateUtil(0, n-1, idx, val, 0);
}

void updateUtil(int l, int r, int idx, int val,
                int i){
  if(l==r){
    tree[i] = val;
    return;
  }
  int mid = (l+r)/2
  if(idx <= mid)
    updateUtil(l, mid, idx, val, 2*i+1);
  else
    updateUtil(mid+1, r, idx, val, 2*i+2);
  tree[i] = tree[2*i+1] + tree[2*i+2];
  }
}
```

Example:

arr = [1, 3, 5, 7, 9, 11]

- Query (1, 3) : 3 + 5 + 7 = 15
- Update (1, 10) → arr = [1, 10, 5, 7, 9, 11]

Date. / /

Lazy Propagation

Used to perform range updates efficiently in segment Trees.

⇒ Problem: You want to add a value v to all elements in a range [l, r]. Naive update is O(n), but lazy propagation does it in O(log n).

⇒ How it Works:

- Use a lazy[] array to store delayed updates
- Propagate updates when visiting nodes (only when necessary).

⇒ Time: O(log(n)) for both query and update

Fenwick Tree (Binary Indexed Tree - BIT)

A compact and efficient data structure for range queries and point updates.

⇒ Use Case:

- Prefix sum queries
- Point updates

⇒ Core Idea:
Each index stores sum of previous elements, cleverly designed with binary operations.

Date. / /

⇒ Time Complexity

Operation	Time
Update	O(log n)
Prefix Query	O(log n)
Space	O(log n)

⇒ Java Code for Fenwick Tree

```
class FenwickTree{
    int[] bit;
    int n;

    FenwickTree(int size){
        n = size;
        bit = new int[n+1];
    }
    void update(int idx, int delta){
        while(idx <= n){
            bit[idx] += delta;
            idx += idx & -idx
        }
    }
    int query(int idx){
        int sum = 0;
        while(idx > 0){
            sum += bit[idx];
            idx -= idx & -idx;
        }
        return sum;
    }
```

```
int rangeQuery(int l, int r){
    return query(r) - query(l-1);
  }
}
```

⇒ Example:- arr = [0, 1, 3, 4, 8, 6, 1, 4, 2] (1-based)

- update(3, +2) → add 2 to index 3
- query(5) → sum of 5 Elements

Summary Table :-

Structure	Range Query	Point Update	Space	Best For
Segment Tree	O(log n)	O(log n)	O(4n)	Range min/max/sum
Lazy segment	O(log n)	O(log n)	O(4n)	Range updat + range queri
Fenwick Tree	O(log n)	O(log n)	O(4n)	Prefix sum & point upd

Date. __/__/____

Interview & Contest - Specific Topics

Java Collections Framework (JCF)

The Java Collections Framework is a unified architecture for storing and manipulating groups of data. It contains interfaces, implementations (classes), and algorithms.

⇒ Common Interfaces

Interface	Description
List	Ordered collection (ArrayList, LinkedList)
Set	Unique elements (HashSet, TreeSet)
Queue	FIFO (LinkedList, PriorityQueue)
Map	Key-value pairs (HashMap, TreeMap)

⇒ Common Implementations

Type	Implementation	Properties
List	ArrayList	Dynamic array, random access
	LinkedList	Doubly linked list

Date. __/__/__

Set	HashSet	Unordered, fast lookup
	TreeSet	Ordered set (BST-based)
Map	HashMap	Unordered key-value store
	TreeMap	Sorted key-value pairs
Queue	Priority Queue	Min/Max-heap behaviour

⇒ Example: Using HashMap

```
Map<String, Integer> map = new HashMap<>();
map.put("apple", 2);
map.put("banana", 4);

System.out.println(map.get("apple")); //output: 2
```

⇒ Time and Space Complexity Analysis

Why It Matters

- Essential for evaluating performance of code
- Crucial during interviews to justify your approach

⇒ Common Complexities

Complexity	Meaning
$O(1)$	Constant time
$O(\log n)$	Logarithmic (Binary search)
$O(n)$	Linear
$O(n \log n)$	Merge Sort, Quick Sort
$O(n^2)$	Nested Loops

Page No

Date. __/__/____

Binary Search on Answer

Apply binary search on the search space, not just the array.

- Example: Minimum speed to deliver packages in k hours

Prefix Sum / Difference Array

Use to pre-compute data for fast range queries.

Hashing

Use HashMap, HashSet for frequency counting, uniqueness etc.

Common Mistakes & Debugging Tricks

@SYNTAX ERROR
—Abhishek Rathor

✗ Mistakes	Fix / Tip ✓
off-by-one errors	check loop boundaries
Infinite loops	Check exit conditions
Using == with strings	Use .equal() in java
Not handling edge cases	Always test with empty, one-elemen
Wrong input/output format	Read the problem carefully

Debugging Tricks

- Use print statements for step-by-step tracing

Date. / /

- Test edge case : emply arrays, max constraints
- Dry Run your code with small input before submitting
- Use custom test cases if allowed in IDE / Contest platform

Summary Table

Topic	Key use case
Java Collections	Fast data storage & access
Time/Space complexity	Analyze and optimize solutions
Sliding Window / 2 Pointers	Array, String problems
Debugging & Testing	Fix logic, performance, and edge issues